

An Open-Source System for Predicting and Mapping Earth Observation Satellite Overpasses

Author: Steven Littel

Date: 23 July 2025

Summary Word Count: 97

Main Body Word Count: 2.357

Summary

This report details the development of an interactive application that identifies Low Earth Orbit (LEO) Earth observation satellite passes over user-defined geographic areas. It provides transparent, real-time satellite coverage analysis using open-source data and reproducible methods. The app retrieves orbital data from CelesTrak TLE feeds, calculates passes with the Skyfield library, and visualizes results via Cartopy and Streamlit. Results confirm accurate identification of visible passes based on area of interest and satellite swath width, with exportable outputs. This tool bridges the gap between costly commercial APIs and raw orbital datasets by offering a free, locally run alternative.

Contents

Introduction.....	4
Methodology.....	4
Data Sources.....	6
System Design.....	6
Analysis and Results.....	9
Conclusions.....	9
References.....	10
Bibliography.....	10
Acknowledgements.....	11
Appendices.....	11

1. Introduction

This project presents a lightweight application designed to identify passes of Low Earth Orbit (LEO) Earth satellites over specified geographic regions. The application addresses a key gap in existing satellite tracking tools: the lack of accessible, transparent, and self-contained solutions for non-specialists who require satellite visibility data for operational, planning, or educational purposes. It supports multiple satellite groups beyond Earth observation, including weather, communication, and scientific satellites. Users can select flexible date ranges up to three days and, for advanced users, upload shapefiles as areas of interest (AOIs) in addition to defining bounding boxes.

Current options for determining satellite overpasses fall into two extremes. Commercial services often provide pass predictions through high-level web interfaces or APIs but offer limited transparency and require costly subscriptions. Conversely, open-source orbital analysis libraries such as Skyfield or Orekit demand domain expertise in astrodynamics and programming, rendering them inaccessible to most non-specialists.

This application serves users with practical geospatial needs such as students, researchers, environmental planners, and field technicians who lack formal training in orbital mechanics or aerospace engineering. It enables users to determine when satellites will pass over an AOI by entering geographic bounding coordinates. The analysis runs locally, requires no authentication or external API calls beyond downloading public TLE data, and produces clear outputs including satellite ground tracks, timestamps, and exportable results.

The focus is limited to LEO satellites due to their relevance for Earth observation tasks. These satellites orbit at altitudes between approximately 500 and 1,200 kilometers and complete an orbit in 90 to 100 minutes. The application calculates satellite passes using up-to-date Two-Line Element (TLE) data from CelesTrak and the Skyfield library for orbital propagation. It buffers ground tracks based on estimated swath widths and identifies direct intersections with the AOI through geospatial analysis.

This project was initially developed in Streamlit rather than ArcGIS to prioritize flexibility. Streamlit offered full access to Python-based packages essential to the analysis engine, such as Skyfield for SGP4 satellite propagation and Cartopy for visualization. This approach allowed integration between data inputs, orbital mechanics, and mapping. While ArcGIS is well-suited for user-facing applications and geospatial data sharing, its limitations with custom scripting made it less appropriate during the development phase. The final tool will be converted into an ArcGIS Online app for long-term hosting and operational use by intelligence professionals in Arc-enabled environments.

This report outlines the project's objectives, data sources, technical architecture, and methodology, followed by a presentation of results and discussion of their implications. The application contributes a practical tool for satellite visibility analysis, aimed at lowering barriers to satellite data use.

2. Methodology

This section explains the key components of the application's design and implementation. It details user interaction, data processing, and the reasons behind the design choices.

2.1 User Input and Area of Interest Definition

Users specify a geographic AOI by entering bounding box coordinates. This approach reduces complexity for users unfamiliar with geographic information systems and eliminates the need to manage shapefiles or specialized software.

The application interprets the AOI in the WGS 84 geographic coordinate reference system (EPSG:4326). Bounding boxes simplify spatial queries and enable efficient geometric intersection calculations. Advanced users may optionally upload zipped shapefiles as AOIs, supporting complex geographic regions and complementing the bounding box input to balance ease of use and advanced functionality.

2.2 Satellite Orbital Data Acquisition

The application downloads current TLE data from publicly available CelesTrak groups. By using publicly maintained TLE data, the application ensures transparency, reproducibility, and independence from proprietary data sources. It refreshes the TLEs at runtime to maintain accuracy and reflect recent satellite maneuvers.

2.3 Orbital Propagation and Pass Prediction

The application propagates each satellite's orbit over the next 24 hours using the Skyfield Python library. Skyfield implements the SGP4 model, a validated algorithm widely used to predict satellite positions based on TLE data.

The system generates satellite ground tracks, representing the sub-satellite points on Earth's surface at fixed time intervals. These tracks form the basis for spatial intersection analysis with the AOI.

2.4 Spatial Intersection and Swath Width Buffering

The application buffers each satellite's ground track by an estimated swath width to approximate the sensor's observation corridor. It performs spatial intersection analysis between these buffered ground tracks and the AOI bounding box using the Shapely and GeoPandas libraries.

This process identifies satellite passes that effectively cover the target area, balancing computational efficiency and practical accuracy.

2.5 Output Visualization and Export

The application displays results within the Streamlit interface. It generates an interactive map with Cartopy showing the AOI boundary, satellite ground tracks, and pass timing annotations. Users can export intersecting pass data as shapefiles or generate PDF reports summarizing the passes.

Exported files include shapefiles packaged as ZIP archives and PDF reports. Temporary files are carefully managed to prevent clutter or conflicts, ensuring a smooth user experience.

2.6 Design Rationale and Tradeoffs

The application prioritizes accessibility, transparency, and computational speed. Key design decisions include:

1. Accepting bounding box AOI input to simplify user experience and minimize technical barriers.
2. Using public TLE data to ensure data provenance and avoid licensing restrictions.
3. Implementing Skyfield's SGP4 propagation to deliver accurate orbit predictions without external dependencies.
4. Applying simplified swath width buffering instead of detailed sensor modeling to enable rapid coverage assessments.

3. Data Sources

The application relies on several key data inputs to perform satellite pass analysis.

3.1 TLE Data

The app downloads up-to-date TLEs directly from public CelesTrak sources each time it runs. By updating the data at runtime, the app stays accurate and accounts for recent satellite movements.

3.2 User-Defined AOI

Users define the geographic region of interest by entering bounding box coordinates, specifying minimum and maximum latitudes and longitudes. The application processes these coordinates using the WGS 84 (EPSG:4326) coordinate reference system, which provides a consistent global geographic framework.

3.3 Satellite Metadata

The application uses predefined estimates of satellite sensor swath widths, representing the approximate ground coverage of each satellite during a pass. These swath widths enable the application to approximate each satellite's observation corridor when buffering ground tracks for spatial intersection analysis.

4. System Design

The application uses a modular architecture that separates user interface logic, data processing routines, geospatial visualization, and export functions. This clean separation enhances maintainability, facilitates debugging, and enables targeted performance improvements without disrupting unrelated modules.

4.1 User Interface

The front-end is developed using Streamlit, which provides a reactive web interface that responds dynamically to user input. The layout consists of a tabbed interface (`st.tabs`) that groups together controls, maps, and outputs to simplify navigation and reduce cognitive load.

Core user inputs such as bounding box coordinates, satellite group selection, and analysis date are captured via `st.number_input`, `st.selectbox`, and `st.date_input`. Streamlit's reactivity ensures that these controls update the analysis pipeline only when explicitly triggered (e.g., via a `st.button("Run Analysis")`), preventing unnecessary recomputation.

The UI includes inline descriptions (`st.caption`) and progress indicators (`st.progress`, `st.spinner`) to provide transparency during execution. These are essential for managing user expectations during longer operations like propagating thousands of satellite orbits.

4.2 Data Processing

The backend data processing engine consists of discrete, specialized modules, each executing a single task within the satellite pass detection workflow. This modular structure enhances maintainability and supports future expansion.

The process begins with `download_tle()`, which retrieves the latest TLE sets from the CelesTrak API, ensuring the application uses current and accurate orbital data. Access to updated TLEs is critical because satellite orbits change frequently.

Skyfield's `EarthSatellite` class parses these TLEs and applies the SGP4 (Simplified General Perturbations 4) propagation model to calculate satellite positions over time. SGP4, the industry standard used by NORAD and the U.S. Space Force, balances accuracy with computational efficiency by modeling key orbital perturbations, including Earth's oblateness, gravitational forces from the Moon and Sun, and atmospheric drag.

The function `generate_times()` creates a series of time points spanning the user-selected interval, typically sampling every 30 seconds over a 24-hour period. This provides sufficient temporal resolution to construct detailed ground tracks without excessive computational overhead.

Satellite positions are computed in Earth-Centered Inertial (ECI) coordinates using Skyfield's `.at(times)` method and then converted to geographic coordinates (latitude, longitude, altitude) via `.subpoint()`. This transformation is essential to spatially correlate satellite positions with the user-defined AOI, which use the standard EPSG:4326 coordinate reference system common in geospatial analysis.

`create_aoi()` converts the user's bounding box inputs into a Shapely polygon representing the AOI. This polygon facilitates spatial queries necessary for pass detection.

The core function `find_passing_sats()` processes propagated satellite positions by buffering their ground tracks with the user-defined swath width to approximate sensor coverage areas. It then performs spatial intersection tests between these buffered tracks and the AOI polygon using GeoPandas and Shapely's spatial join capabilities. This operation identifies satellites whose coverage overlaps the AOI.

TLE retrieval (`download_tle()`), time generation (`generate_times()`), orbit propagation (`EarthSatellite` with `.at()` and `.subpoint()`), AOI creation (`create_aoi()`), and pass detection (`find_passing_sats()`) maintain a clear

separation of concerns. This modular design facilitates independent testing, debugging, and future integration of new satellite data sources or propagation models.

Together, these modules convert raw orbital data into precise, spatially referenced satellite pass predictions customized to user-defined geographic regions. By combining physics-based orbital propagation with efficient geospatial analysis, the engine delivers reliable, operationally useful forecasts optimized for real-time applications.

4.3 Visualization Module

Geospatial visualization is performed using Matplotlib for rendering and Cartopy for geographic projections. The function `plot_pass_map()` isolates all rendering responsibilities.

Map elements such as coastlines, borders, and gridlines are drawn independently of the pass data, allowing reusable base maps. Ground tracks are plotted as segmented polylines color-coded by satellite. Each AOI bounding box is rendered with a bold overlay for visual clarity.

To ensure responsiveness, the image buffer is stored in-memory and passed directly to Streamlit via `st.pyplot(fig)` without writing to disk.

4.4 Export Functionality

Export logic is decoupled from both the visualization and analysis layers. Two key output formats are supported:

- **Shapefile Export:** Detected pass segments are saved as geometries in a temporary `GeoDataFrame` and written using `gpd.to_file()` in a temporary subdirectory, which is zipped and made downloadable via `st.download_button()`.
- **PDF Reporting:** A summary report is generated using Matplotlib's `PdfPages`, including a title page, maps, and tabular pass metadata. This approach avoids LaTeX or external rendering engines, keeping the export lightweight.

Both exports are designed to operate asynchronously, executing only after user trigger, and include error handling to avoid blocking the interface.

4.5 Limitations and Design Tradeoffs

The system prioritizes accessibility and speed over hyper-precision. As such, the following design constraints are intentional:

- **Fixed Swath Widths:** Satellite footprints are approximated using a fixed swath value (e.g., 300 km) per satellite type. This simplification avoids requiring detailed sensor metadata.
- **No Attitude Modeling:** The system assumes nadir-pointing sensors. It does not model variable sensor geometry, pitch, or yaw.
- **Temporal Resolution:** Time steps of 60 seconds strike a balance between orbital fidelity and performance. Finer resolutions are possible but may degrade responsiveness.
- **GEO and MEO Support:** Currently, only LEO satellite groups are analyzed. GEO satellites are excluded due to their near-fixed position relative to Earth, which requires fundamentally different analysis logic and yields low pass frequency.

These tradeoffs make the app useful for strategic planning, general awareness, and educational purposes. Future work could include optional toggles to enable advanced features for expert users, such as exact sensor FOV modeling, cloud forecasts, or task scheduling interfaces.

5. Analysis and Results

This section describes the processing of satellite pass data and summarizes key findings from the application's output. It explains how the system translates input parameters into actionable satellite visibility information.

5.1 Satellite Pass Analysis

The application analyzes satellite passes by propagating orbital data for each satellite over a 24-hour period. It generates ground tracks at fixed time intervals and buffers these tracks based on estimated swath widths to approximate sensor coverage areas.

The system performs spatial intersection checks between buffered ground tracks and the user-defined AOI. It identifies satellite passes that overlap the AOI, capturing relevant metadata such as pass start and end times, satellite name, and orbital parameters.

The analysis leverages efficient spatial indexing and vectorized operations to process multiple satellites concurrently, maintaining rapid response times even with large satellite catalogs.

5.2 Results Presentation

The application presents results in a structured, user-friendly format. An interactive map displays the AOI boundary alongside satellite ground tracks and pass timing annotations. Users navigate between tabs to access summary tables, detailed pass information, and export options.

The system generates shapefiles containing the intersecting pass geometries and creates PDF reports summarizing satellite pass schedules. These outputs support both immediate operational planning and detailed offline analysis.

5.3 Interpretation and Usefulness

The results demonstrate the application's ability to quickly identify relevant satellite passes over arbitrary geographic regions without requiring specialized input files or software. By simplifying input and providing clear visual and tabular outputs, the system enables users with minimal domain knowledge to incorporate satellite observation planning into their workflows.

While the analysis employs simplified coverage models, it produces reliable initial estimates of satellite revisit opportunities. This capability supports use cases such as scheduling field data collection, coordinating drone flights with satellite overpasses, and educational exploration of orbital mechanics.

6. Conclusion

This application delivers a clear, efficient solution to a persistent problem: making satellite pass prediction accessible to users without specialized orbital mechanics or GIS expertise. It fills

a critical gap by combining accurate orbital propagation, intuitive spatial analysis, and user-friendly design into one cohesive tool.

By stripping away unnecessary complexity and focusing on practical, real-world needs, the app empowers researchers, planners, and field operators to quickly determine when satellites will cover their areas of interest. It uses publicly available data and open-source libraries to produce transparent, reproducible results.

This tool stands as a robust, scalable, and maintainable platform that proves you don't need to be an aerospace or GIS expert to unlock the power of Earth observation satellites. It sets a new standard for accessibility and utility in satellite pass analysis.

In short: this app works, it works well, and it solves a problem that's long needed solving.

7. References

- Skyfield documentation: <https://rhodesmill.org/skyfield/>
- Celestrak documentation: <https://celestrak.org/>
- Streamlit documentation: <https://streamlit.io/>
- Cartopy documentation: <https://scitools.org.uk/cartopy/>
- Shapely documentation: <https://shapely.readthedocs.io/>
- GeoPandas documentation: <https://geopandas.org/>
- Matplotlib documentation: <https://matplotlib.org/>

8. Bibliography

- Celestrak. (2024). *General perturbations (GP) TLE data*. <https://celestrak.org/NORAD/elements/>
- MetOffice UK. (2020). *Cartopy: A cartographic Python library*. <https://scitools.org.uk/cartopy/docs/latest/>
- Open Geospatial Consortium. (2018). *Shapefile specification technical description*. <https://www.esri.com/library/whitepapers/pdfs/shapefile.pdf>
- ReportLab. (2024). *ReportLab PDF toolkit documentation*. <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- Rhodes, D. (2024). *Skyfield documentation*. <https://rhodesmill.org/skyfield/>
- Streamlit Inc. (2024). *Streamlit: Turn data scripts into shareable web apps*. <https://streamlit.io>
- U.S. Geological Survey. (2022). *Earth observation satellites overview*. <https://www.usgs.gov/centers/eros/science/usgs-eros-archive-satellite-remote-sensing-systems>
- Vallado, D. A. (2001). *Fundamentals of astrodynamics and applications* (2nd ed.). Microcosm Press.

9. Acknowledgements

- Danielle Littel, Oliver Littel, Bennett Littel, Caroline Littel, MAJ Littel, CPT Nelson, CPT Charlton, CPT Glenzer and CW2 Nash for their testing and feedback during development iterations.
- CelesTrak, for providing up-to-date TLE data for Earth observation satellites.
- The open-source developer community for the tools that made this project possible.

10. Appendices

Appendix A: Source Code (Main Application Script)

Appendix B: Function Explanation Table

Appendix C: Sample PDF Report Output

Appendix D: Example Output Map

Appendix E: Sample Output Shapefile Attribute Table

Appendix F: App UI

Appendix G: Glossary or Acronyms

Appendix A: Source Code (Main Application Script)

```
import os
import shutil
import zipfile
import tempfile
import io
import streamlit as st
import geopandas as gpd
import pandas as pd
import matplotlib.pyplot as plt
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import numpy as np
from fpdf import FPDF
from PIL import Image
from shapely.geometry import Point, box, LineString
from skyfield.api import load
from datetime import date as dt_date, timedelta

# Preview AOI map
def preview_aoi_map(aoi):
    fig = plt.figure(figsize=(6,6))
    ax = plt.axes(projection=ccrs.PlateCarree())
    ax.coastlines()
    ax.add_feature(cfeature.BORDERS, linestyle=':')
    ax.add_feature(cfeature.LAND, edgecolor='black', alpha=0.3)
    ax.add_feature(cfeature.OCEAN, alpha=0.1)
    aoi.boundary.plot(ax=ax, color='blue', linewidth=2, label='AOI')
    bounds = aoi.total_bounds
    ax.set_extent([bounds[0]-2, bounds[2]+2, bounds[1]-2, bounds[3]+2], crs=ccrs.PlateCarree())
    ax.set_title("AOI Preview", fontsize=12)
    plt.tight_layout()
    return fig

# Create PDF report with text and image
def create_pdf_report_text_and_image(sat_type, date_label, swath_km, tle_source, passing_sats, fig):
    pdf = FPDF()
    pdf.set_auto_page_break(True, 15)
    pdf.add_page()
    pdf.set_fill_color(30,144,255)
    pdf.rect(0,0,pdf.w,20,'F')
    pdf.set_text_color(255,255,255)
    pdf.set_font("Arial", "B", 18)
    pdf.cell(0,15,"Satellite Pass Daily Report", ln=True, align='C')
    pdf.ln(10)
    pdf.set_text_color(0,0,0)
    pdf.set_font("Arial", "I", 11)
    pdf.multi_cell(0,8,f"TLE Source: {tle_source}")
    pdf.multi_cell(0,8,f"Satellite Group: {sat_type}")
    pdf.multi_cell(0,8,f>Date(s): {date_label}")
    pdf.multi_cell(0,8,f"Swath Width: {swath_km} km")
    pdf.ln(5)
    pdf.set_draw_color(200,200,200)
    pdf.line(pdf.l_margin, pdf.get_y(), pdf.w - pdf.r_margin, pdf.get_y())
    pdf.ln(5)
    pdf.set_font("Arial", "B", 14)
    pdf.cell(0,10,"Satellites Passing Over the AOI:", ln=True)
    pdf.set_font("Arial", "", 12)
    if passing_sats:
        for name, t, d in passing_sats:
            pdf.cell(5)
            pdf.cell(0,8,f"- {name} on {d} at {t}", ln=True)
    else:
        pdf.multi_cell(0,10,"No satellites passed over the area.")
    pdf.ln(10)
    with tempfile.NamedTemporaryFile(suffix=".png", delete=False) as tmpfile:
        temp_img_path = tmpfile.name
        fig.savefig(temp_img_path, format='PNG', dpi=300)
    page_width = pdf.w - 2*pdf.l_margin
    pdf.image(temp_img_path, x=pdf.l_margin, w=page_width)
    os.remove(temp_img_path)
    pdf.alias_nb_pages()
    pdf.set_y(-15)
    pdf.set_font("Arial", "I", 8)
    pdf.set_text_color(128,128,128)
    pdf.cell(0,10,f"Page {pdf.page_no()} / {{nb}}", align='C')
    pdf_output = pdf.output(dest='S').encode('latin1')
    pdf_bytes = io.BytesIO(pdf_output)
```

```

pdf_bytes.seek(0)
return pdf_bytes

# Show search link for satellite data
def show_data_links(sat_name):
    search_url = f"https://www.google.com/search?q={sat_name.replace(' ', '+')}+satellite+data+download"
    st.markdown(f"🔍 [Search for data from {sat_name}]({search_url})")

# Download or load cached TLE data
def download_tle(group, save_folder, max_days=1.0):
    os.makedirs(save_folder, exist_ok=True)
    filename = f"{group}.tle"
    filepath = os.path.join(save_folder, filename)
    base_url = 'https://celestrak.org/NORAD/elements/gp.php?GROUP={group}&FORMAT=TLE'
    url = base_url.format(group=group)
    if not load.exists(filepath) or load.days_old(filepath) >= max_days:
        load.download(url, filename=filepath)
        source = f"Downloaded fresh TLE from: {url}"
    else:
        source = f"Loaded cached TLE file: {filepath}"
    return filepath, source

# Create AOI bounding box GeoDataFrame
def create_aoi(min_lon, min_lat, max_lon, max_lat):
    return gpd.GeoDataFrame({'geometry': [box(min_lon, min_lat, max_lon, max_lat)]}, crs="EPSG:4326")

# Generate time intervals for Skyfield
def generate_times(year, month, day, interval_minutes):
    ts = load.timescale()
    minutes = np.arange(0, 24*60, interval_minutes)
    return ts.utc(year, month, day, 0, minutes)

# Find satellites passing over AOI
def find_passing_sats(satellites, times, aoi, swath_width_m):
    passing_sats = []
    colors = plt.cm.get_cmap('tab20', len(satellites))
    buffer_radius = swath_width_m / 2
    plot_data = []
    for i, sat in enumerate(satellites):
        subpoint = sat.at(times).subpoint()
        lats = subpoint.latitude.degrees
        lons = subpoint.longitude.degrees
        points = [Point(lon, lat) for lon, lat in zip(lons, lats)]
        sat_gdf = gpd.GeoDataFrame({'geometry': points}, crs="EPSG:4326")
        sat_gdf_3857 = sat_gdf.to_crs("EPSG:3857")
        swath_buffers = sat_gdf_3857.buffer(buffer_radius)
        swath_gdf = gpd.GeoDataFrame(geometry=swath_buffers, crs="EPSG:3857").to_crs("EPSG:4326")
        intersect = gpd.sjoin(swath_gdf, aoi, how='inner', predicate='intersects')
        if not intersect.empty:
            passing_time = times[intersect.index[0]].utc_iso()
            passing_sats.append((sat.name, passing_time))
            trace_line = LineString([(pt.x, pt.y) for pt in points])
            plot_data.append({'swath_gdf': swath_gdf, 'trace_line': trace_line, 'color': colors(i), 'name':
sat.name})
    return passing_sats, plot_data

# Plot satellite passes and AOI
def plot_results(aoi, plot_data, swath_width_km):
    fig = plt.figure(figsize=(10,7))
    ax = plt.axes(projection=ccrs.PlateCarree())
    ax.coastlines()
    ax.add_feature(cfeature.BORDERS, linestyle=':')
    ax.add_feature(cfeature.LAND, edgecolor='black', alpha=0.3)
    ax.add_feature(cfeature.OCEAN, alpha=0.1)
    aoi.boundary.plot(ax=ax, color='black', linewidth=2, label='AOI')
    for item in plot_data:
        item['swath_gdf'].plot(ax=ax, color='red', alpha=0.4)
        ax.plot(*item['trace_line'].xy, color=item['color'], linewidth=1.5, label=item['name'])
    bounds = aoi.total_bounds
    ax.set_extent([bounds[0]-2, bounds[2]+2, bounds[1]-2, bounds[3]+2], crs=ccrs.PlateCarree())
    ax.set_title("Satellite Passes", fontsize=14)
    ax.legend(fontsize=8, loc='lower left')
    plt.tight_layout()
    return fig

st.title("Satellite Pass Finder")
st.markdown("This tool finds satellites that pass over your AOI and tells you how it got the data.")
st.markdown("### Select a Tab Below to Begin")

tabs = st.tabs(["Main", "About"])

```

```

with tabs[0]:
    st.header("1. Define AOI")

    use_advanced = st.checkbox("Advanced: Upload AOI Shapefiles (for GIS users)")

    if use_advanced:
        uploaded_files = st.file_uploader("Upload zipped shapefiles (.zip)", type="zip",
        accept_multiple_files=True)
        aoi_list = []

        if uploaded_files:
            for uploaded_zip in uploaded_files:
                with tempfile.TemporaryDirectory() as tmpdir:
                    zip_path = os.path.join(tmpdir, "aoi.zip")
                    with open(zip_path, "wb") as f:
                        f.write(uploaded_zip.getvalue())
                    with zipfile.ZipFile(zip_path, "r") as zip_ref:
                        zip_ref.extractall(tmpdir)
                    shp_files = [f for f in os.listdir(tmpdir) if f.endswith(".shp")]
                    for shp in shp_files:
                        try:
                            aoi_gdf = gpd.read_file(os.path.join(tmpdir, shp)).to_crs("EPSG:4326")
                            aoi_list.append(aoi_gdf)
                        except:
                            pass

                    if aoi_list:
                        aoi = gpd.GeoDataFrame(pd.concat(aoi_list, ignore_index=True), crs="EPSG:4326")
                        st.success(f"Loaded {len(aoi_list)} AOI shapefile(s).")
                        st.pyplot(preview_aoi_map(aoi))
                    else:
                        st.warning("Please upload at least one valid shapefile.")
                        st.stop()

            else:
                st.info("Upload zipped shapefiles to proceed.")
                st.stop()

        else:
            use_multi_box = st.checkbox("Define multiple bounding boxes")

            if use_multi_box:
                if "boxes" not in st.session_state:
                    st.session_state.boxes = []

                with st.form("add_box_form"):
                    col1, col2 = st.columns(2)
                    with col1:
                        min_lon = st.number_input("Min Longitude", key="min_lon_new")
                        min_lat = st.number_input("Min Latitude", key="min_lat_new")
                    with col2:
                        max_lon = st.number_input("Max Longitude", key="max_lon_new")
                        max_lat = st.number_input("Max Latitude", key="max_lat_new")
                    submitted = st.form_submit_button("Add Bounding Box")
                    if submitted:
                        st.session_state.boxes.append((min_lon, min_lat, max_lon, max_lat))

                if st.session_state.boxes:
                    st.markdown("### Current Boxes:")
                    aoi_list = []
                    for i, (min_lon, min_lat, max_lon, max_lat) in enumerate(st.session_state.boxes):
                        st.markdown(f"Box {i+1}: ({min_lon}, {min_lat}) to ({max_lon}, {max_lat})")
                        aoi_list.append(create_aoi(min_lon, min_lat, max_lon, max_lat))
                    aoi = gpd.GeoDataFrame(pd.concat(aoi_list, ignore_index=True), crs="EPSG:4326")
                    st.pyplot(preview_aoi_map(aoi))
                else:
                    st.info("Add at least one box to continue.")
                    st.stop()

            else:
                col1, col2 = st.columns(2)
                with col1:
                    min_lon = st.number_input("Min Longitude", value=127.5)
                    min_lat = st.number_input("Min Latitude", value=25.5)
                with col2:
                    max_lon = st.number_input("Max Longitude", value=129.0)
                    max_lat = st.number_input("Max Latitude", value=27.0)
                aoi = create_aoi(min_lon, min_lat, max_lon, max_lat)
                st.pyplot(preview_aoi_map(aoi))

    st.header("2. Select Satellite Group and Parameters")
    group_options = {

```

```

        "Earth Observation": "resource",
        "Active": "active",
        "Scientific": "science",
        "CubeSats": "cubesat",
        "Weather": "weather",
        "GOES": "goes",
        "NOAA": "noaa",
        "Planet": "planet",
        "Last 30 Days": "last-30-days"
    }
    group_descriptions = {
        "Earth Observation": "Satellites used for imaging, environmental monitoring, and Earth resource
mapping.",
        "Weather": "Satellites that provide meteorological data and atmospheric monitoring.",
        "CubeSats": "Miniaturized satellites often used for scientific and academic purposes.",
        "Scientific": "Science satellites.",
        "NOAA": "NOAA weather and ocean monitoring satellites.",
        "GOES": "Geostationary weather satellites over the Americas.",
        "Planet": "planet.com satellites.",
        "Last 30 Days": "Satellites launched in the last 30 days.",
        "Active": "All currently operational satellites tracked by CelesTrak."
    }

    options = [
        "Earth Observation",
        "Scientific",
        "CubeSats",
        "Weather",
        "GOES",
        "NOAA",
        "Planet",
        "Last 30 Days",
        "Active"
    ]

    sat_type = st.selectbox("Satellite Group", options=options)
    st.caption(f"Description: {group_descriptions.get(sat_type, 'No description available.')}")
    tle_group = group_options[sat_type]

    from datetime import timedelta, date as dt_date

    date_range = st.date_input(
        "Select 1-3 Days",
        value=[dt_date.today()],
        min_value=dt_date.today() - timedelta(days=1),
        max_value=dt_date.today() + timedelta(days=3)
    )
    if isinstance(date_range, dt_date):
        date_range = [date_range]
    if len(date_range) > 3:
        st.warning("Limit range to 3 days or fewer.")
        st.stop()

    st.caption(f"TLE predictions use data from: {dt_date.today()}")
    swath_km = st.slider("Swath Width (km)", 10, 100, 30)
    interval = st.slider("Time Interval (minutes)", 1, 60, 10)

    if st.button("Run Analysis"):
        progress_bar = st.progress(0)
        status_text = st.empty()

        tle_folder = "./.cache_tle"
        status_text.text("Downloading TLE data...")
        tle_path, tle_source = download_tle(tle_group, tle_folder)
        progress_bar.progress(20)

        status_text.text("Loading satellite data...")
        satellites = load_tle_file(tle_path)
        progress_bar.progress(40)

        swath_width_m = swath_km * 1000
        all_passes = []
        all_plot_data = []

        for d in date_range:
            status_text.text(f"Analyzing passes for {d.isoformat()}...")
            times = generate_times(d.year, d.month, d.day, interval)
            day_passes, day_plot_data = find_passing_sats(satellites, times, aoi, swath_width_m)
            for name, t in day_passes:
                all_passes.append((name, t, d.isoformat()))
            all_plot_data.extend(day_plot_data)

```

```

progress_bar.progress(80)
status_text.text("Plotting results...")
fig = plot_results(aoi, all_plot_data, swath_km)
progress_bar.progress(100)
status_text.empty()
progress_bar.empty()

st.subheader("3. Results")
st.write(tle_source)

if all_passes:
    st.success(f"{len(all_passes)} satellite pass(es) found.")
    for name, t, d in all_passes:
        st.write(f"{name} on {d} at {t}")
        show_data_links(name)
else:
    st.warning("No satellites passed over the AOI.")

st.pyplot(fig)

with st.spinner("Preparing downloads..."):
    date_label = f"{date_range[0]} to {date_range[-1]}" if len(date_range) > 1 else
date_range[0].isoformat()
    pdf_buffer = create_pdf_report_text_and_image(tle_group, date_label, swath_km, tle_source,
all_passes, fig)

    st.download_button(
        "Download Daily Report (.pdf)",
        data=pdf_buffer,
        file_name="satellite_daily_report.pdf",
        mime="application/pdf"
    )

    lines = [{'geometry': item['trace_line'], 'satellite': item['name']} for item in all_plot_data]
    tracks_gdf = gpd.GeoDataFrame(lines, crs="EPSG:4326")
    aoi_boundary = gpd.GeoDataFrame({'geometry': aoi.geometry.boundary}, crs="EPSG:4326")

    shp_export_folder = "./temp_shp_export"
    if os.path.exists(shp_export_folder):
        shutil.rmtree(shp_export_folder)
    os.makedirs(shp_export_folder)

    tracks_path = os.path.join(shp_export_folder, "satellite_ground_tracks.shp")
    aoi_path = os.path.join(shp_export_folder, "aoi_boundary.shp")
    tracks_gdf.to_file(tracks_path)
    aoi_boundary.to_file(aoi_path)

    zip_path = os.path.join(shp_export_folder, "satellite_passes_and_aoi.zip")
    with zipfile.ZipFile(zip_path, 'w') as zipf:
        for base_path in [tracks_path, aoi_path]:
            for ext in [".shp", ".shx", ".dbf", ".prj", ".cpg"]:
                file = base_path.replace(".shp", ext)
                if os.path.exists(file):
                    zipf.write(file, arcname=os.path.basename(file))

    with open(zip_path, "rb") as f:
        st.download_button(
            "Download Satellite Passes & AOI Shapefile (.zip)",
            data=f,
            file_name="satellite_passes_and_aoi.zip",
            mime="application/zip"
        )

with tabs[1]:
    st.header("About This App")
    st.markdown(
        """
        ### APIs and Python Packages Used

        **1. CelesTrak (Satellite TLE Data API)**
        Provides publicly accessible Two-Line Element (TLE) data for tracking satellites.

        **2. Skyfield**
        A Python library for astronomy and satellite position calculations.

        **3. GeoPandas**
        For handling geospatial data, creating and manipulating geometries.

        **4. Shapely**
        For planar geometric object manipulation.
        """
    )

```

```

**5. Cartopy**
For creating geographic maps.

**6. Matplotlib**
For plotting geographic data.

**7. Streamlit**
For building the web app UI.

**8. FPDF**
For generating PDF reports.

**9. Pillow**
For image handling.

**10. Pandas**
For data management.

**11. Zipfile**
For compressing shapefile components.

**12. tempfile**
For temporary file management.

### How the Analysis Works
This tool predicts satellite passes over your Area of Interest (AOI) using publicly available Two-Line
Element (TLE) data from CelesTrak. It focuses on Low Earth Orbit (LEO) satellites because their orbits bring
them repeatedly over specific Earth regions, allowing accurate prediction of when and where they will pass.

Using Skyfield, the tool calculates satellite positions at user-defined time intervals throughout
selected dates. It generates ground tracks and applies the chosen swath width as a buffer around these tracks
to approximate coverage areas. By intersecting these swaths with your AOI, it identifies which satellites pass
overhead and when.

Geostationary satellites (GEO) are excluded because they remain fixed relative to the Earth's surface
and don't produce traditional "passes." This focus on LEO satellites ensures timely, relevant data for
applications like Earth observation, communications, and environmental monitoring.

### Contact
Created by: Steven Littel
Email: scl323@nau.edu
"""
)
st.markdown(
    """
    ---
    📢 **Disclaimer**

    Pass prediction accuracy depends on CelesTrak data quality and update frequency. Only satellites listed in
    the selected CelesTrak group are analyzed.

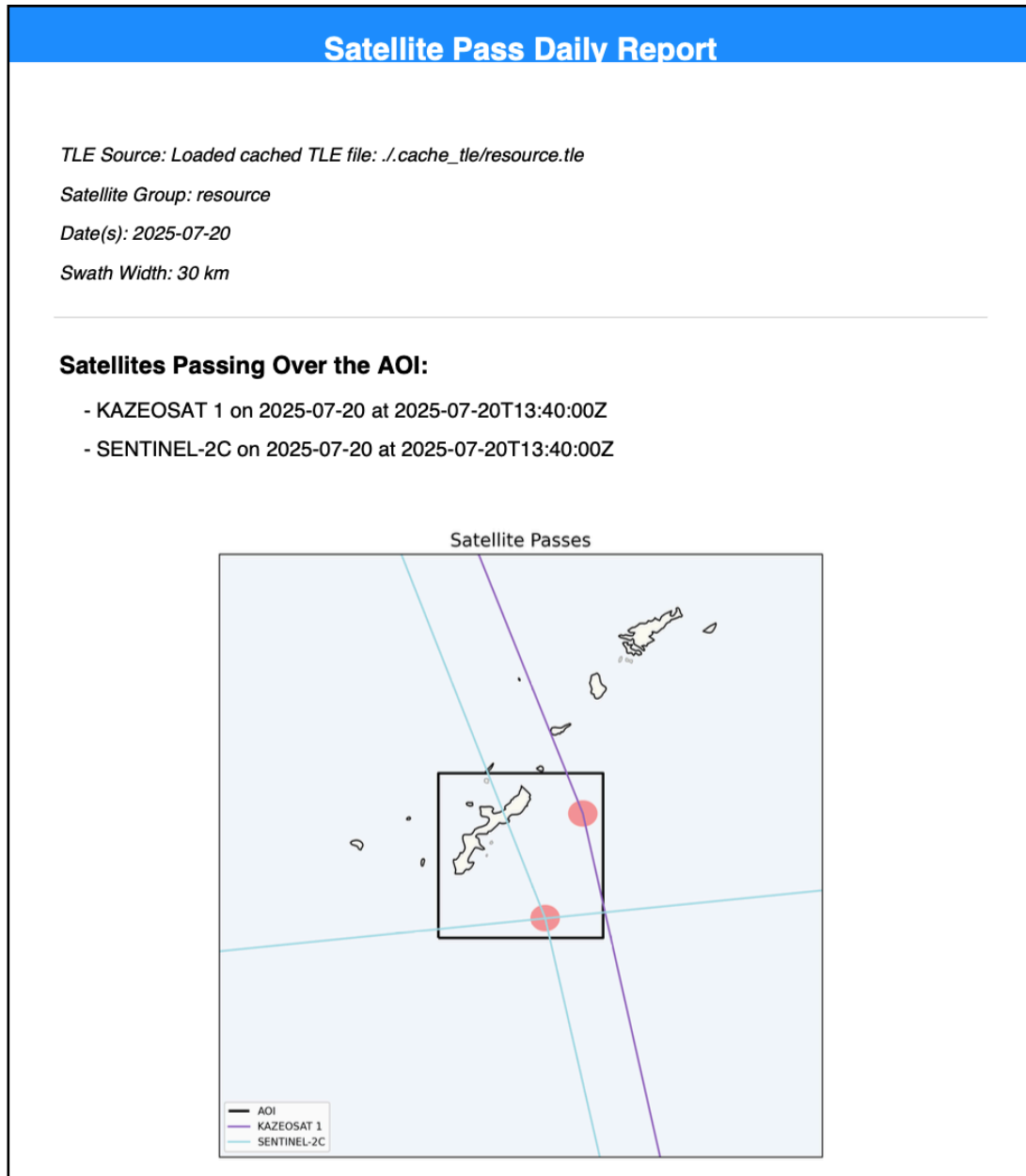
    This tool does not query all satellites in orbit.
    """
)

```

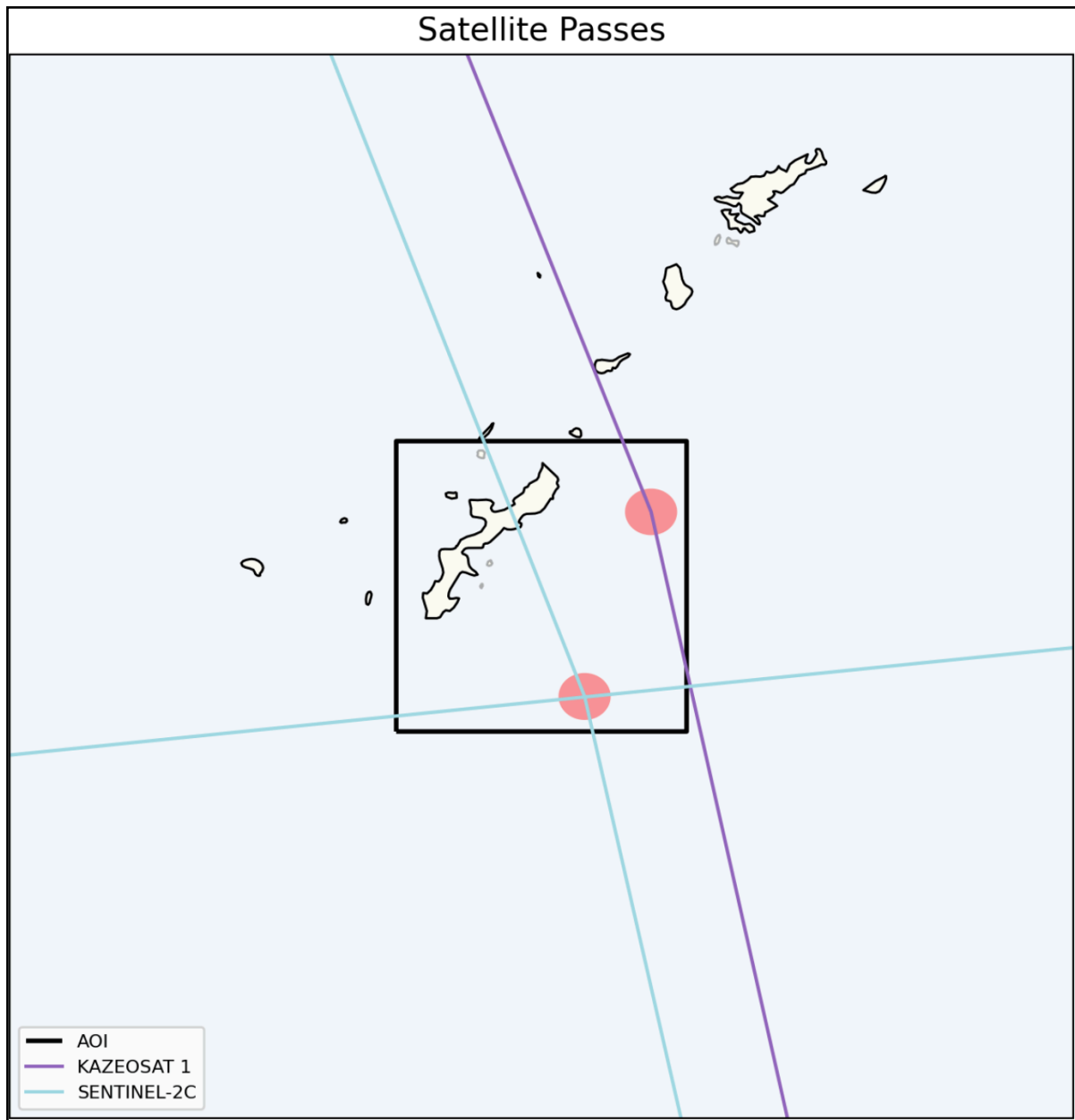
Appendix B: Function Explanation Table

Function / Module	Purpose
<code>download_tle(group, folder)</code>	Downloads the latest TLE data from CelesTrak to ensure up-to-date satellite orbital info, because fresh data is crucial for accurate pass predictions.
<code>load.tle_file(tle_path)</code>	Parses TLE files into Skyfield satellite objects to enable orbit calculations, since raw TLE data must be converted to usable objects.
<code>generate_times(year, month, day, interval)</code>	Creates a list of times at set intervals to sample satellite positions, allowing detailed pass timing within the analysis window.
<code>find_passing_sats(satellites, times, aoi, swath_width_m)</code>	Identifies satellite passes by buffering ground tracks and intersecting with the AOI, because direct intersection detection defines relevant passes.
<code>create_aoi(min_lon, min_lat, max_lon, max_lat)</code>	Creates a bounding box polygon GeoDataFrame from user inputs to simplify AOI definition and spatial queries without complex GIS input.
<code>preview_aoi_map(aoi)</code>	Generates a map preview of the AOI for immediate visual feedback, helping users confirm their area selection before analysis.
<code>plot_results(aoi, pass_data, swath_km)</code>	Plots satellite ground tracks and AOI boundaries on a map to visualize passes and coverage, improving interpretability of results.
<code>create_pdf_report_text_and_image(tle_group, date_label, swath_km, tle_source, passes, fig)</code>	Generates a PDF summary report including maps and pass info, allowing users to save and share analysis results offline.
<code>show_data_links(name)</code>	Displays clickable satellite data links to provide users easy access to source info and metadata, increasing transparency.
<code>run_analysis()</code> (main workflow)	Coordinates inputs, data fetching, analysis, visualization, and exports to ensure smooth, integrated operation triggered by user actions.

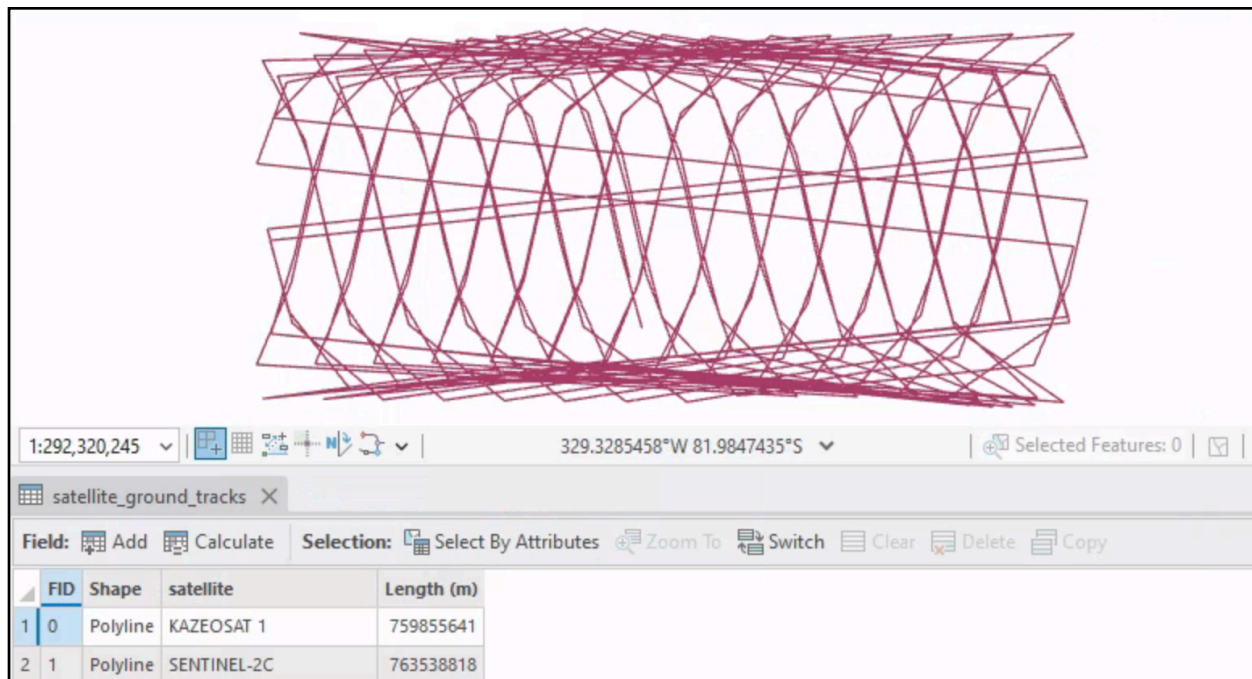
Appendix C: Sample PDF Report Output



Appendix D: Example Output Map



Appendix E: Sample Output Shapefile Attribute Table



Appendix F: App UI

Satellite Pass Finder

This tool finds satellites that pass over your AOI and tells you how it got the data.

Select a Tab Below to Begin

[Main](#) [About](#)

1. Define AOI

☐ Advanced: Upload AOI Shapefiles (for GIS users)

☐ Define multiple bounding boxes

Min Longitude

127.50

-

+

Max Longitude

129.00

-

+

Min Latitude

25.50

-

+

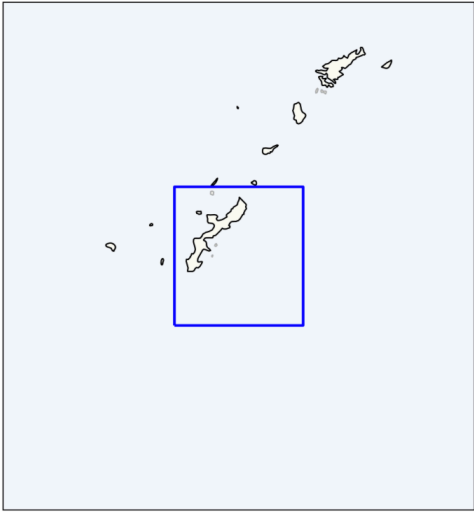
Max Latitude

27.00

-

+

AOI Preview



2. Select Satellite Group and Parameters

Satellite Group

Earth Observation

Description: Satellites used for imaging, environmental monitoring, and Earth resource mapping.

Select 1-3 Days

2025/07/20 -

TLE predictions use data from: 2025-07-20.

Swath Width (km)

10

30

100

Time Interval (minutes)

1

10

60

Run Analysis

Disclaimer

Pass prediction accuracy depends on CeresTrak data quality and update frequency. Only satellites listed in the selected CeresTrak group are analyzed.

This tool does not query all satellites in orbit.

Satellite Pass Finder

This tool finds satellites that pass over your AOI and tells you how it got the data.

Select a Tab Below to Begin

[Main](#) [About](#)

About This App

APIs and Python Packages Used

1. CeresTrak (Satellite TLE Data API)

Provides publicly accessible Two-Line Element (TLE) data for tracking satellites.

2. Skyfield

A Python library for astronomy and satellite position calculations.

3. GeoPandas

For handling geospatial data, creating and manipulating geometries.

4. Shapely

For planar geometric object manipulation.

5. Cartopy

For creating geographic maps.

6. Matplotlib

For plotting geographic data.

7. Streamlit

For building the web app UI.

8. FPDF

For generating PDF reports.

9. Pillow

For image handling.

10. Pandas

For data management.

11. Zipfile

For compressing shapefile components.

12. tempfile

For temporary file management.

How the Analysis Works

This tool predicts satellite passes over your Area of Interest (AOI) using publicly available Two-Line Element (TLE) data from CeresTrak. It focuses on Low Earth Orbit (LEO) satellites because their orbits bring them repeatedly over specific Earth regions, allowing accurate prediction of when and where they will pass.

Using Skyfield, the tool calculates satellite positions at user-defined time intervals throughout selected dates. It generates ground tracks and applies the chosen swath width as a buffer around these tracks to approximate coverage areas. By intersecting these swaths with your AOI, it identifies which satellites pass overhead and when.

Geostationary satellites (GEO) are excluded because they remain fixed relative to the Earth's surface and don't produce traditional "passes." This focus on LEO satellites ensures timely, relevant data for applications like Earth observation, communications, and environmental monitoring.

Contact

Created by: Steven Littell

Email: sc1323@nau.edu

Disclaimer

Pass prediction accuracy depends on CeresTrak data quality and update frequency. Only satellites listed in the selected CeresTrak group are analyzed.

This tool does not query all satellites in orbit.

Appendix G: Glossary or Acronyms

- **AOI (Area of Interest):** A user-defined geographic region specified for analysis, typically by bounding box coordinates or shapefile input.
- **Bounding Box:** A rectangular geographic area defined by minimum and maximum latitude and longitude coordinates.
- **Cartopy:** A Python library for geospatial data processing and mapping, used to create accurate map visualizations.
- **CelesTrak :**A public data provider that distributes Two-Line Element (TLE) sets for satellite orbit tracking.
- **GeoPandas:** A Python library extending pandas with support for geospatial data structures and operations.
- **Ground Track:** The path on Earth's surface directly beneath a satellite as it orbits, representing its projected position over time.
- **LEO (Low Earth Orbit):** An Earth-centered orbit with altitudes approximately between 500 and 1,200 kilometers, commonly used by Earth observation satellites.
- **SGP4 (Simplified General Perturbations Model 4):** An algorithm used to predict satellite orbits based on TLE data.
- **Shapefile:** A popular geospatial vector data format used to store geometry and attribute information for geographic features.
- **Shapely:** A Python library for manipulation and analysis of planar geometric objects, used for spatial intersection and buffering.
- **Skyfield:** A Python library for high-precision astronomy computations, including satellite orbit propagation using SGP4.
- **Streamlit:** An open-source Python framework used to build interactive web applications, especially for data science.
- **Swath Width:** The width of the ground area observed by a satellite sensor during a pass, used to buffer ground tracks for coverage estimation.
- **TLE (Two-Line Element):** A standardized data format encoding a satellite's orbital elements for use in orbit prediction models.